Model Behavior II

March 2026

Model Behavior II

1 Project Overview

This project aims to predict California house prices using machine learning models with increasing levels of complexity. The target variable is LogClosePrice, which reduces skewness and improves model stability.

We compare Two models XGBoost Model and LightGBM Model. .

Model performance is evaluated using R^2 (log scale and original dollar scale) to measure explained variance. MAPE to measure average percentage prediction error. MdAPE to measure typical (median) percentage error, which is more robust to outliers.

2 XGBoost

2.1 Baseline XGBoost Model

Gradient Boosting Tree model (XGBoost Regressor) builds many decision trees sequentially, where each new tree corrects the errors of the previous ones. We built a baseline XGBoost model to predict the LogClosePrice of California Houses.

Feature Selection

- LivingArea
- BathroomsTotalIntege
- BedroomsTotal
- BedBath$_Ratio$
- PropertyAge
- MonthsFromDec2025
- SqftPerBedroom
- LotUtilization
- ZipMedianPrice: obtained by taking the median of each postal code and assigning it to each row of the dataset by postal code.

Metric	Training Set	Testing Set
R^2 (log scale)	0.929	0.894
R^2 (actual scale)	0.900	0.833
MAPE (%)	12.280%	14.523%
MdAPE (%)	9.056%	10.248%

Figure 1: Performance of XGBoost Model

Dependent Variable: LogClosePrice.
Model Performance
High R^2: The model explains 93% of variance on training data and 89% on unseen test data, indicating that most of the variation in the target variable cannot be explained by the model.
Low MAPE and MdAPE: The mean and median absolute percentage errors are low, meaning predictions are, on average, within 9-15% of the true property prices.
Robustness: The difference between training and testing MAPE, MdAPE, and suggests the model is slightly overfitting and handles unseen data decently well.
Practical Use: This XGBoost model is effective at producing accurate predictions of residential property closing prices, as indicated by the low MAPE and MdAPE values. However, the relatively low suggests that the model explains only a limited portion of the variation in the target variable LogClosePrice, meaning that important factors influencing property prices may not be captured by the current feature set.

2.2 Hyperparameter Tuning

We applies hyperparameter tuning to the baseline XGBoost model. RandomizedSearchCV is used to test different combinations of hyperparameters and find the best-performing model based on cross-validation R^2. After running the randomized search, the best combination of hyperparameters is selected based on cross-validation performance.

Best Parameters: 'subsample': 0.7, 'nestimators': 500, 'minchildweight': 5, 'maxdepth': 5, 'learningrate': 0.1, 'gamma': 0, 'colsamplebytree': 0.7 Best CV Score: 0.9039463438660847

	Metric	Training Set	Testing Set
0	R^2 (log)	0.924370	0.896200
1	R^2 (actual)	0.891650	0.839739
2	MAPE (%)	12.612172	14.438064
3	MdAPE (%)	9.321650	10.190544

Figure 2: Performance of tuned XGBoost Model

Interpretation of Tuned Model Results: The tuned XGBoost model is evaluated on both the log scale and the actual price scale.
R^2 on the log scale shows how well the model explains variation in LogClosePrice. R^2 on the actual scale shows how well the back-transformed predictions match real home prices. MAPE and MdAPE measure percentage error on the actual ClosePrice scale, which makes them more interpretable for housing prices.

A small gap between training and testing metrics suggests that the tuned model generalizes reasonably well.

3 LightGBM

3.1 Baselin LightGBM Model

Model Features The treefeatures list includes:

- Location (ZipMedianPrice)
- Property attributes (LivingArea, BathroomsTotalInteger, BedroomsTotal)
- Engineered features (BedBathRatio, SqftPerBedroom, LotUtilization)
- Time/age features (PropertyAge, MonthsFromDec2025)

Instead of using one-hot encoding for ZIP codes, we encode each ZIP by the median log price of homes in that ZIP code, which captures neighborhood-level pricing effects and significantly reduces dimensionality. For ZIP codes that appear in the test set but not in the training set, we assign the global median log price.
LightGBM Regressor Gradient boosting model (LGBMRegressor) trains a sequence of decision trees to minimize prediction error. We built the baseline LightGBM with the following Parameters: nestimators=200, number of trees. maxdepth=6, max depth per tree to avoid overfitting. learningrate=0.1, step size for each boosting iteration.

LightGBM Model Performance

Metric	Training Set	Testing Set
R^2	0.916	0.890
MAPE (%)	13.24	14.84
MdAPE (%)	9.69	10.50

Figure 3: Baseline LightGBM Performance

High R^2: The model explains 92% of variance on training data and 89% on unseen test data, indicating strong predictive power and reasonable generalization.
Low MAPE and MdAPE: The mean and median absolute percentage errors are low, meaning predictions are, on average, within 15% of the true property prices, with typical errors closer to 10%.

Robustness: The small difference between training and testing metrics suggests the model is not overfitting and handles unseen data well.
Practical Use: This LightGBM model effectively captures non-linear relationships between property features and prices, making it suitable for real estate price estimation and decision-making.

Overall, the LightGBM model provides an accurate, interpretable, and robust approach for predicting residential property closing prices using feature-engineered data.

3.2 Hyperparameter Tuning for LightGBM

The baseline LightGBM model above uses a fixed set of parameters. That gives us a strong starting point, but it does not tell us whether the model is too simple, too complex, or leaving performance on the table.

In this section, we tune the model to find a better balance between bias, variance, and generalization. We use cross-validation so that the selected hyperparameters are judged on multiple validation folds instead of one lucky split. This makes the tuning process more reliable.

Hyperparameter tuning was performed using RandomizedSearchCV with 3-fold cross-validation to efficiently search for improved LightGBM parameters while keeping computational cost manageable. Instead of exhaustively evaluating all possible combinations as in GridSearchCV, randomized search samples combinations from the parameter space, allowing us to explore important parameters while reducing training time.

The best hyperparameters identified were:

- learningrate = 0.05
- nestimators = 200
- numleaves = 150
- maxdepth = -1
- minchildsamples = 10
- subsample = 0.9
- colsamplebytree = 0.8
- regalpha = 5
- reglambda = 1

These parameters balance model complexity and generalization.

Model Performance
R^2: The test R^2 value of 0.8925 indicates that the model explains nearly 89% of the variance in the test dataset, suggesting strong predictive performance. The relatively small gap between training and testing R^2 suggests that the model generalizes well and is not severely overfitting.
MAPE MdAPE The error metrics also provide insight into prediction accuracy. A test MAPE of 14.58% means that predictions differ from actual values by about 14.6% on average, while

Metric	Training Set	Testing Set
R^2	0.9205	0.8925
MAPE (%)	12.75	14.58
MdAPE (%)	9.21	10.12

Figure 4: Tuned LightGBM Performance

the median absolute percentage error (MdAPE) of 10.12% indicates that most predictions have even smaller errors. The difference between mean and median errors suggests that the majority of predictions are relatively accurate, with a small number of larger errors.

Overall, hyperparameter tuning improved the LightGBM model by identifying a parameter configuration that achieves a strong balance between predictive accuracy and generalization on unseen data.

4 Model Performance Comparison

Metric	Linear Regression (Train)	Linear Regression (Test)	XGBoost (Train)	XGBoost (Test)	LightGBM (Train)	LightGBM (Test)
R^2 (log)	0.762	0.727	0.9243	0.8949	0.9205	0.8925
MAPE (%)	22.16	23.90	12.60	14.50	12.75	14.58
MdAPE (%)	15.91	17.08	9.29	10.29	9.21	10.12

Figure 5: Model Performance Comparison

4.1 Model Performance Analysis

Linear Regression

Linear regression underperforms compared to tree-based models, especially for non-linear relationships.

Strength: Simple and interpretable. Reasonably consistent between train and test sets (R^2 drops only from 0.762 → 0.727).

Weaknesses: Lowest R^2 values among all models, meaning it explains less variance in the target. Highest MAPE (22.16% train, 23.90% test) and MdAPE (15.91% train, 17.08% test), indicating larger errors.

XGBoost

Strengths: Highest R^2 on training (0.9243) and very strong on testing (0.8949), showing excellent predictive power. Lowest MAPE (12.60% train, 14.50% test) and MdAPE (9.29% train, 10.29% test), meaning more accurate predictions. Handles non-linear patterns and interactions effectively.

Weaknesses: Slight overfitting: R^2 drops from 0.9243 → 0.8949, though the drop is not ex-

treme. Less interpretable compared to linear regression.

LightGBM
Strengths: Comparable performance to XGBoost (R^2: 0.9205 train, 0.8925 test). MAPE and MdAPE very similar to XGBoost (slightly higher on test set). Generally faster and more memory-efficient than XGBoost on large datasets.
Weaknesses: Slightly worse on test set than XGBoost (R^2 drops a bit more, errors slightly higher). Like XGBoost, it is a complex model and less interpretable.

So the best overall model is XGBoost, due to high R^2 and lowest error metrics. Close competitor: LightGBM, slightly less accurate but faster. Least suitable: Linear Regression, mainly due to lower predictive power and higher errors.